

Certified but Compromised: Breaking and Fixing Lightweight Proofs of Inference

Final Report — Trustworthy Machine Learning, Spring 2026

Ofek Lavi
Tel Aviv University
Tel Aviv, Israel
ofeklavi@mail.tau.ac.il

Erel Barzilay
Tel Aviv University
Tel Aviv, Israel
erelbarzilay@mail.tau.ac.il

Edo Kadosh
Tel Aviv University
Tel Aviv, Israel
edo@mail.tau.ac.il

Abstract

A company that outsources inference to a cloud provider cannot tell whether the provider ran the agreed model or changed the answer on purpose. Proofs of inference let it check each answer. Those built on zkSNARKs give strong guarantees but take minutes per query on large models, which has motivated lighter protocols that check only a sample of the computation, such as that of Anchuri et al., in which the client re-checks one random path through a committed execution trace. We show that sampling misses targeted changes: a provider that tampers with a single neuron and computes everything else honestly almost always passes the path check, on image classifiers and on a real large language model, where one neuron is enough to change the next token. No sampling rule, however it chooses where to look, closes this gap on every network. We therefore build a protocol that checks every layer with Freivalds’ algorithm against a Reed–Solomon/Merkle commitment to the weights. It needs no SNARK, trusted hardware or trusted setup, a wrong output passes with negligible probability, and in our experiments it rejected thousands of attacks without a miss. Its prover is orders of magnitude faster than published zk-SNARK provers and its verifier is competitive with theirs; its main cost is a proof that grows with the input.

Keywords

verifiable inference, proof of inference, integrity attacks, Freivalds’ algorithm, zkML

1 Introduction

Consider a company that pays a cloud provider to serve a licensed or fine-tuned language model to its users. Nothing in a response shows that the provider ran that model with the agreed weights. The provider may cut costs by serving a cheaper quantised copy, but it may also return a wrong answer on purpose. A company that handles sensitive data and acts on the model’s answers, for example to approve payments, flag fraud or triage medical records, is exposed to a provider, or an attacker inside it, that changes the answers to chosen queries and serves every other query correctly. The company cannot re-run the model to find out, since it lacks the weights and the hardware; that is why it outsourced inference. A *proof of inference* closes this gap. With each answer the provider sends a proof, and the client (here, the company) runs a check, much cheaper than inference, that fails with high probability unless the answer was computed with the agreed weights on this input.

zkSNARKs [8, 15, 25, 29, 31] prove that the answer is correct and can keep the weights secret. Their proofs are usually small and fast to check, but producing one takes far longer than inference; zkLLM needs about ten minutes on an A100 for one Llama-2-7B prompt [31]. Anchuri et al. [2] proposed a much lighter approach. The provider commits to the full execution trace with a Merkle tree, and the client checks the computation along *one random path* from an output neuron back to the input, against a commitment to the weights. Proving drops from minutes to milliseconds. A different model produces a trace that differs from the correct one in many places, and a random path is likely to hit one of them. Anchuri et al. accordingly prove *other-model* soundness, against a provider that honestly runs a different model. They note that one path cannot catch a single changed node with probability above $1/N$ in a layer of width N , and they leave *full* soundness, against any wrong answer, open for LLMs.

We present an attack on this protocol, which we implement and evaluate. A provider that keeps the committed model and changes only what it needs to can overwrite *a single neuron* and recompute everything after it honestly (Figures 1a and 1b). The trace is then inconsistent at one node, which a random path visits with probability $1/N$, so almost every tampered answer is accepted. Applied only to inputs that carry a trigger, the edit is a backdoor in the serving code, which leaves the committed weights clean and answers clean queries bit for bit as before. The attack carries over to the real OPT-6.7B. Checking many queries does not close the gap, because the provider chooses which queries to tamper with, and tampering with k of them is caught with probability at most k/N . Nor do the fixes Anchuri et al. suggest, sampling more paths or sampling where it matters: uniform sampling is already optimal among rules that ignore the claimed values, no rule that reads them does better on every network, the rules we tested still miss almost every attack, and reaching 2^{-40} forces the path protocol to open essentially the whole model and trace.

This led us to a different design (Figure 1c). The client tests one random linear combination of all the neurons of every layer (Freivalds’ algorithm [14]). When it does not hold the weights, it checks the combination against a Reed–Solomon-encoded, Merkle-committed copy of them in the style of Ligerio [1], so, as in the path protocol, it stores 32 bytes per weight matrix. The model runs as an int8 network whose results are bit-identical on the provider’s GPU and on the client, which makes the test exact. A tampered value anywhere in the network is then caught with probability $1 - 2^{-\lambda}$ instead of $1/N$, and the provider’s extra work grows with the model but not with the input. At 2^{-128} our proofs are also smaller than

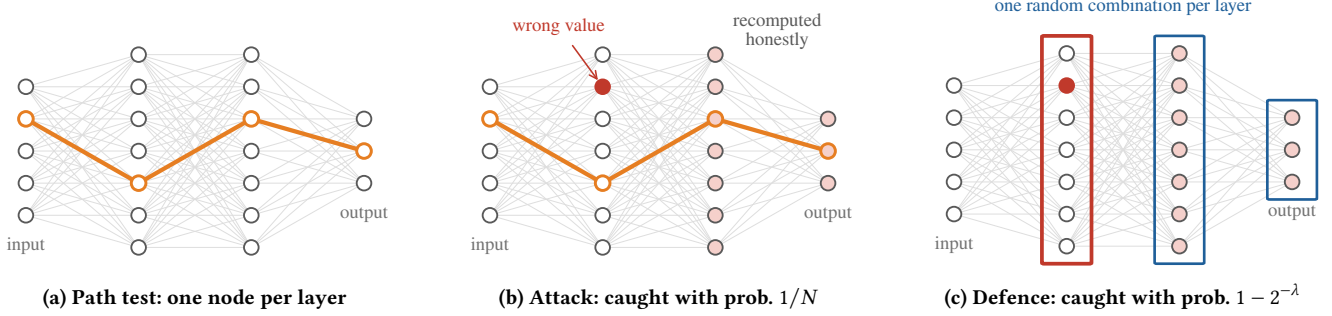


Figure 1: One tampered neuron slips past a random path, but not past a check of every layer

the paths’ at 2^{-40} : catching the single-neuron attack on a 64-token Llama-2-7B prompt with probability $1 - 2^{-40}$ takes paths that open about 13.7 GB, while our proof is 329 MB. An exact check of a few neurons leaves the rest of the network unconstrained; one random combination per layer constrains all of it.

Contributions. (1) **An attack** (Sections 3.2 and 4.2) that exploits the $1/N$ limit acknowledged in [2]. One tampered neuron is accepted with probability $1 - 1/N$ (99.6–99.8% on MNIST), works as a trigger backdoor that leaves the weights and clean answers unchanged, and changes the next token of 40 of 40 prompts on the real OPT-6.7B. (2) **A bound on sampling-based fixes** (Section 3.3): no sampler that ignores the claimed values beats uniform sampling, no sampler at all does so on every network, and uniform sampling needs essentially the whole model and trace for 2^{-40} ; a sampler that weights each parent by its contribution is blind to zeroed neurons. (3) **A full-soundness protocol** for whole CNNs and transformers (Sections 3.4–3.6), with error $2^{-\lambda}$, in the setting of [2]: it composes known checks, and to our knowledge it is the first that needs only a hash commitment per weight matrix, with no SNARK, TEE or trusted setup (Section 2). Optimisations keep the bound and shrink the committed-weights proof 2.0–4.2× for images and prompts of up to 512 tokens. (4) **A matched-model evaluation** (Section 4) on six image classifiers and 10 LLM architectures up to 13B parameters, against [2], 17 published systems and zkLLM’s code on our GPU. Our prover is 30–32,000× faster than the zkSNARK provers. Run non-interactively, the optimised protocol also beats zkCNN on LeNet-5 and DeepProve’s transparent configuration on 64-token GPT-2 prompts on all three costs, with the caveats of Section 4.6. Code and raw measurements are released.

2 Related Work

zkSNARK proofs of inference. A long line of work compiles neural networks into zero-knowledge proofs. For CNNs these include sum-check/GKR provers [17] such as zkCNN [25] and zkPyTorch [35], and pairing- or Plonk-based systems such as vCNN [23], ZEN [11], ZENO [12], ZKML [8], EZKL [39] and Bionetta [38]; Mystique [34] is an interactive, VOLE-based alternative. For LLMs they include zkLLM [31] (also interactive), zkGPT [29], DeepProve [15], ZKTorch [7] and Jolt Atlas [4]. They give computational soundness and mostly small proofs, but their

provers are far slower than inference: from half a second for LeNet-5 to minutes or hours for larger models.

Freivalds-style checks. SafetyNets [16] uses interactive proofs for networks with quadratic activations. Slalom [33] checks every linear layer of a GPU computation from inside an SGX enclave with Freivalds’ algorithm, in exact arithmetic modulo a 24-bit prime and optionally with a precomputed secret challenge; TwinShield [36] and VeriAttn [9] extend this to attention inside TEEs. Several recent works need no enclave. Maverick [26] checks every linear layer of a language model with sparse Freivalds checks on a code-encoded copy of the weights that the client stores, drawing a fresh challenge per query. Gupta et al. [19] run Freivalds’ check in the exponent of a group, against verification data that the model owner generates with secret randomness, for a client that outsources inference to two non-colluding servers. CommitLLM [6] checks int8 language models with secret Freivalds vectors precomputed from the weights, audits sampled positions and layers, and leaves attention unverified. LAMP [22] lets a verifier that holds only a commitment check one matrix product, using Freivalds’ check and Reed–Solomon proximity testing inside a Groth16 SNARK.

Sampling, fingerprints and optimistic protocols. Anchuri et al. [2] spot-check random paths and argue security through trace separation; SLP [24] samples which layers receive a SNARK. TOPLOC [28] and SVIP [32] fingerprint hidden states, with empirical guarantees. Verde [3], opML [10] and TAO [37] resolve disputes and assume an honest party or an active challenger.

Where we fit. We keep the setting of [2], a client that holds a 32-byte commitment per weight matrix, but replace its path test, which we break (Sections 3.2 and 3.3), with full soundness $2^{-\lambda}$ for complete CNNs and transformers (Sections 3.4 and 3.5). We chain known protocols through every layer with an end-to-end soundness proof: Freivalds’ check, and the column check of the Ligero and Brakedown commitments [1, 18]. The protocols above that check every layer without a SNARK each need more than a hash commitment: Slalom an enclave, Maverick an encoded copy of the weights, CommitLLM a secret key derived from them, and Gupta et al. public data that grows with every matrix and is generated with secret randomness. zkSNARKs that verify against a Merkle root, such as DeepProve’s transparent configuration, need no trusted setup but prove far more slowly. To our knowledge, ours is the first protocol that is fully sound for complete CNNs and transformers

using only a hash function and a linear code, with no SNARK, TEE or trusted setup.

3 Technical Approach

We fix the setting, break the path protocol (Sections 3.2–3.3), and then build a protocol that checks every layer (Sections 3.4–3.6).

3.1 Setting and threat model

A model M has a public architecture and secret or public weights. The provider acts as the prover and the client as the verifier. Before any query the client obtains a commitment C_M to the weights, for example from the model owner. As in [2], we assume that C_M is computed honestly; unlike a SNARK’s trusted setup it involves no secret randomness, and anyone who holds the weights can recompute it. For each query x the provider returns a claimed output y and a proof, and may deviate from the protocol arbitrarily. We require *completeness* (honest answers are always accepted) and *full soundness*: any $y \neq M(x)$ is rejected except with probability ϵ . Anchuri et al. prove only *other-model soundness*, in which the adversary must honestly run some different model. We consider three trust settings. In setting C the client holds only C_M (32 bytes per weight matrix), as in [2] and the zkSNARKs; in K it knows the weights; and K_{pre} is K with a secret challenge precomputed once per model, as in Slalom’s preprocessing mode and CommitLLM [6]. The architecture includes the integer constants of the operations without weights (requantisation multipliers, look-up tables), which the client obtains with C_M .

3.2 The path test and the single-neuron attack

In the protocol of [2] (RandPathTest) the prover commits to every activation a_j of the forward pass with a Merkle tree. The verifier chooses a random output neuron and walks back to the input, moving to a random parent at each layer. At every node j on the path it opens the parents’ activations and the weights w_{ij} from C_M and checks $a_j = \phi(\sum_{i \in G_j} w_{ij} a_i)$, where G_j is the set of parents of j and ϕ the activation. A path opens one weight row per layer, so the proof and the prover’s work stay small.

The single-neuron attack. Our adversary runs the correct model on x , picks one neuron j , and replaces a_j by a value that makes the output wrong (untargeted) or equal to a class of its choice (targeted). It then recomputes all later layers *with the true weights*. Every later node is consistent with its parents and every earlier node is untouched, so j is the only node whose local check fails, the only *inconsistent node*. The verifier notices the attack only if its path visits j , which happens with probability $1/N$ in a layer of width N , or $1 - (1 - 1/N)^s$ with s paths. Applied only to inputs that contain a trigger, the attack becomes a backdoor in the serving code. The prover keeps the committed weights and answers clean queries honestly, bit for bit, so no inspection of the weights reveals it and the clean-accuracy gap is zero by construction.

3.3 Why sampling does not fix it

Anchuri et al. suggest sampling more paths, or sampling where it matters instead of uniformly. Neither helps against an adversary who knows the rule. Fix a layer of width N and let U be the set

of its neurons whose change can flip the output. If the sampler ignores the claimed values, the probabilities with which it opens each neuron are fixed before the adversary acts and sum to at most s over U , so the adversary can tamper with a neuron that is opened with probability at most $s/|U|$, on every network. A sampler that reads the values could do better on a particular network, but not on every one. We call a sampler *value-aware* if it chooses each neuron to open from the input, the claimed values, the weights of later layers, each neuron’s natural range (the values it takes on honest inputs) and the rows it has already opened.

PROPOSITION 3.1 (NO SAMPLER BEATS UNIFORM ON EVERY NETWORK). *For every value-aware sampler that opens at most s neurons of a hidden layer of width N and accepts every network’s honest trace, there is a network on which it catches a tampered neuron, kept within its natural range, with probability at most s/N .*

Proof. Let hidden neuron i copy input bit $\sigma(i)$ for a permutation σ , let natural inputs have at most one zero, and let the output be 1 exactly when all hidden neurons are 1. The adversary sets the neuron that copies the input’s zero to 1, which flips the output and stays in the range $\{0, 1\}$. The forged trace, later weights and ranges do not depend on σ , and the forged trace is honest for the network in which that neuron copies another bit, so the sampler rejects only if it opens the tampered row. Every other row it opens shows a neuron that copies a 1, so for a random σ the zero’s copier stays uniform among the unopened neurons and is opened with probability at most s/N ; some σ does no better. $\square$

Opening s distinct neurons uniformly reaches s/N on every dense network, so nothing beats uniform sampling in the worst case, and catching one tampered neuron with constant probability needs $s = \Omega(N)$ openings per layer, the cost the protocol was designed to avoid. On our MNIST network the *contribution-weighted sampler*, which steps to parent i with probability proportional to $|w_{ij} a_i|$, catches our single-neuron attack, whose value is unusually large, in 9% of paths, but never visits a neuron whose claimed activation is zero, so zeroing a few neurons is never detected. A floor δ , sampling by $|w_{ij}|(|a_i| + \delta)$, removes this blind spot but leaves detection low (Section 4.2), and a range check costs the attack only a few more neurons.

3.4 Our defence: checking every layer

Since no sampler has a better worst-case guarantee than uniform sampling, our verifier checks every neuron of every layer. We first split “the answer is correct” into three statements, then show how the verifier checks each one cheaply, and finally how the checks combine.

Exact integers. The checks compare exact values, but floating-point results differ between devices and summation orders. The model that is committed and checked is therefore an int8 version of M , obtained by integer-only quantisation [21], the form in which many models are already served; Section 4.1 measures its accuracy. Each weight layer ℓ computes $Z_\ell = W_\ell X_\ell + b_\ell$, where W_ℓ is int8 with one scale per output channel, X_ℓ is int8 with one scale per tensor, and BatchNorm is folded into W_ℓ and b_ℓ ; convolutions become matrix products through im2col. These integer products can still use the GPU’s fast float32 units: float32 represents every integer below

2^{24} exactly, so the prover sums at most 1,024 products of int8 values at a time, as in Slalom [33], and its results are bit-identical to the verifier's. Exactness also removes the floating-point tolerance of the path protocol, which leaves an attacker room to hide small changes.

Three statements. With the output y , the prover sends the pre-activations Z_ℓ of every weight layer, which we call the *claims*. Write $A_\ell = [W_\ell | b_\ell]$. Between two weight layers the network applies only operations without weights: requantisation, ReLU, pooling, residual additions, LayerNorm/RMSNorm, GELU/SiLU (by table look-up), RoPE, softmax, and the attention products $QK^\top$ and PV . The answer y is the last weight layer's pre-activations (the logits), so it is itself a claim, and it is correct if every claim is. Every claim equals the value that M computes on x if and only if three statements hold for every weight layer ℓ : (S1) its input X_ℓ follows from x and the earlier claims through these operations, for example $X_\ell = \text{requant}(\text{ReLU}(Z_{\ell-1}))$ in an MLP; (S2) its claims are the product of its weights and its input, $Z_\ell = A_\ell[X_\ell; \mathbf{1}]$; and (S3) A_ℓ is the committed matrix.

Checking (S1): recomputation. The verifier computes every operation without weights itself, from x and the claims. Each input X_ℓ is therefore the verifier's own computation, not a value sent by the prover, and the checks are chained: a prover cannot pass one layer's check with one value and feed another into the next. This is also why non-linear layers such as ReLU and softmax need no proof here: the verifier never checks a claimed activation, it computes it.

Checking (S2): Freivalds' check. Let A_ℓ have N rows (one per output neuron) and row length k , and let T be the number of columns of X_ℓ (pixels or tokens); we drop the subscript ℓ from N , k and T when the layer is clear. Recomputing $A_\ell[X_\ell; \mathbf{1}]$ would cost the verifier as much as inference. Instead it draws r independent random vectors $\chi \in \mathbb{F}_p^N$ and, for each, checks one random combination of the layer's rows,

$$\chi^\top Z_\ell = \underbrace{(\chi^\top A_\ell)}_{u_\ell} [X_\ell; \mathbf{1}], \quad (1)$$

where we call u_ℓ the *folded row*. Let $D = Z_\ell - A_\ell[X_\ell; \mathbf{1}]$ be the error of the claims: its entry D_{im} is how far the claim of neuron i at position m (a pixel or token) is from the true value, so a correct layer has $D = 0$. The two sides of Eq. (1) differ by $\chi^\top D$, so the check passes only if $\chi^\top D = 0$. If some $D_{im} \neq 0$, entry m of $\chi^\top D$ is $\sum_{i'} \chi_{i'} D_{i'm} = \chi_i D_{im} + c$, where c does not involve χ_i . Whatever the other entries of χ are, this is zero for only one of the p values χ_i can take ($\chi_i = -c/D_{im}$), so a wrong layer passes with probability at most $1/p$; it must pass the check for each of the r vectors, so at most p^{-r} in all. This holds however many values are wrong and wherever they are: a single tampered neuron is caught as reliably as a wholly wrong layer. Given u_ℓ , the two sides cost $O(r(N+k)T)$, against $O(NkT)$ for recomputing the layer; u_ℓ itself costs $O(rNk)$, paid by the prover in C and by the verifier, per query in K and once per model in K_{pre} .

We work in the BabyBear field $\mathbb{F}_p$, $p = 15 \cdot 2^{27} + 1 \approx 2^{31}$. Integers map into $\mathbb{F}_p$ without wrap-around as long as they are small, and two checks ensure this. The verifier range-checks every claim ($|z| < 2^{29}$), and setup checks that every honest pre-activation is in

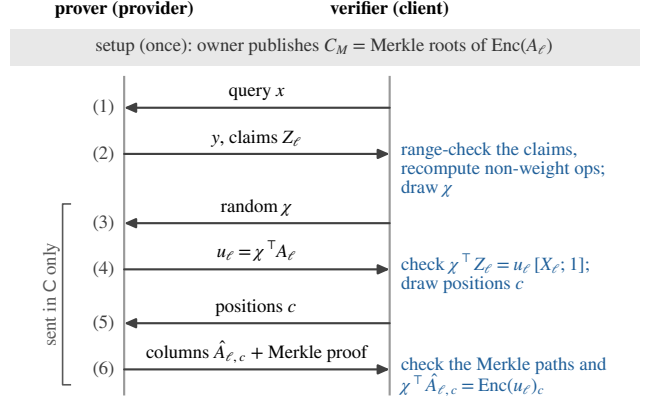


Figure 2: One query in setting C: each value is fixed before the randomness that tests it

the same range: $128 \sum_c |w_{ic}| + |b_i| < 2^{29}$ for every row i of a product, whose inputs are int8 and so at most 128 in magnitude, and $\max_c |w_{ic}| + |b_i| < 2^{29}$ for an embedding, whose claim at each token is one column of its table. A wrong claim and the correct value then differ by less than $2^{30} < p$, so they remain distinct in $\mathbb{F}_p$.

Checking (S3): committed weights. In K and K_{pre} the verifier holds the weights and computes u_ℓ itself, so (S3) holds by construction. In setting C it cannot, so the prover sends u_ℓ , and the verifier must check that $u_\ell = \chi^\top A_\ell$ for the committed A_ℓ . Opening t random entries of u_ℓ against the weights would be weak: a prover who changes one of the k entries escapes with probability $1 - t/k$. We therefore commit to an *encoded* copy of the weights, as in the Ligerio and Brakedown commitments [1, 18]. At setup every row of A_ℓ is Reed–Solomon encoded [30]: the polynomial whose coefficients are the row is evaluated at $n_\ell = 4\bar{k}$ points, where $\bar{k} \geq k$ is a power of two (Section 3.6 relaxes this), giving $\hat{A}_\ell$. The commitment C_M holds one Merkle root [27] per weight matrix, over the columns of $\hat{A}_\ell$. Encoding is linear, so $\chi^\top \hat{A}_\ell = \text{Enc}(\chi^\top A_\ell)$. The verifier encodes the claimed u_ℓ itself, opens t random columns c with their Merkle paths, and checks $\chi^\top \hat{A}_{\ell,c} = \text{Enc}(u_\ell)_c$. If u_ℓ is wrong, $\text{Enc}(u_\ell)$ and $\text{Enc}(\chi^\top A_\ell)$ are distinct codewords whose difference is a nonzero polynomial of degree below k , so they agree on at most $k - 1$ of the n_ℓ positions, and t distinct columns all miss the difference with probability at most $((k - 1)/n_\ell)^t < 4^{-t}$. Because C_M is computed honestly, every committed row is a codeword and no proximity test is needed.

Putting the checks together. Suppose $y \neq M(x)$. Then some claim is wrong, since y is one; let ℓ be the first layer with a wrong claim. Its input X_ℓ is correct, because the verifier computed it from x and correct earlier claims (S1). So $Z_\ell \neq A_\ell[X_\ell; \mathbf{1}]$ for the committed A_ℓ , and the prover can pass only by fooling one of the two random checks: if it sends the true u_ℓ , Eq. (1) fails except with probability p^{-r} (S2); if it sends another u_ℓ , a column check fails except with probability $((k_\ell - 1)/n_\ell)^t$ (S3). Theorem 3.2 adds these terms over the layers.

3.5 Protocol and soundness

Setup happens once per model: the owner encodes and commits the weights and publishes C_M . Figure 2 shows one query of the basic protocol, with r vectors χ and t columns per layer; besides recomputing the operations without weights (S1) and the checks of (S2) and (S3), the verifier range-checks the claims. Because the claims precede χ and u_ℓ precedes the column positions, the prover fixes every value before it sees the randomness that tests it.

THEOREM 3.2 (SOUNDNESS). *If C_M is honest and the prover cannot find a hash collision, then for any claimed $y \neq M(x)$ the verifier accepts with probability at most $\varepsilon = \sum_\ell [p^{-r} + (\frac{k_\ell-1}{n_\ell})^t]$.*

Proof. As shown in Section 3.4, if $y \neq M(x)$ the first layer with a wrong claim passes only if Freivalds’ check or its column check is fooled (the range check keeps the wrong claim distinct from the true value in F_p , and the opened columns are the committed ones unless the prover finds a hash collision), which happens with probability at most p^{-r} or $((k_\ell-1)/n_\ell)^t$. A union bound over the layers finishes the proof. $\square$

Parameters. For a target λ and L weight layers we set $\beta = \lambda + \log_2(2L)$, $r = \lceil \beta / \log_2 p \rceil$ and $t = \lceil \beta / 2 \rceil$; since $n_\ell \geq 4k_\ell$, each of the $2L$ terms of ε is at most $2^{-\beta}$, so $\varepsilon \leq 2^{-\lambda}$ (for VGG-16 at $\lambda=128$, $r=5$ and $t=67$). Interactively, each challenge is expanded with SHAKE-256 from a fresh 256-bit random key; deriving the keys from a hash of the transcript (Fiat-Shamir [13]) makes the protocol non-interactive. Although it has two challenge rounds, the protocol is round-by-round sound [5]: once the claims are fixed, only the first wrong layer’s χ can make them pass (probability at most p^{-r}), and once u_ℓ is fixed, only that layer’s columns can (at most its column term). In the random-oracle model a prover that makes Q hash queries therefore succeeds with probability at most $Q 2^{-\beta}$, plus that of finding a hash collision [20]; with 64 extra bits in β this is below $2^{-\lambda}$ for $Q \leq 2^{64}$.

Known weights. In K the verifier computes u_ℓ itself and messages (3)–(6) are omitted. In K_{pre} it computes the secret χ and u_ℓ once per model, so the proof is the claims alone. The prover never sees the secret χ , but it learns each verdict and, once a forgery is accepted, can repeat it. Up to that point, rejecting every query with a wrong claim does not depend on χ , so the prover’s queries are fixed in advance, and each passes with probability at most $\sum_\ell p^{-r}$: a false acceptance occurs among Q queries that reuse χ with probability at most $Q \sum_\ell p^{-r}$.

3.6 Costs and optimisations

Costs. The prover runs the forward pass, computes u_ℓ and opens t columns, which it recomputes from W_ℓ ; its extra work, $O((r+t) \sum_\ell N_\ell k_\ell)$, does not depend on the number of pixels or tokens. With 4-byte claims and field elements and 32-byte hashes, a proof in setting C has

$$|\pi| = 4 \sum_\ell N_\ell T_\ell + 4r \sum_\ell k_\ell + 4t \sum_\ell N_\ell + 32 \sum_\ell h_\ell \text{ bytes} : (2)$$

the claims, which grow with the input, and the folded rows, the t opened columns of N_ℓ entries and the $h_\ell \leq t \log_2 n_\ell$ hashes of each layer’s Merkle multiproof, which do not. For Llama-2-7B at 64 tokens these are 349, 23, 385 and 4 MB of the basic protocol’s 762 MB

proof. In K and K_{pre} the proof is the claims alone. zkCNN’s and zkLLM’s proofs, by contrast, stay between tens and a few hundred kilobytes.

Optimisations. The parameters above were chosen to keep the soundness argument short, and they leave the proof larger than it needs to be. We call the protocol so far the *basic* protocol, and the one with the four changes below, checked by the engineered verifier described after them, the *optimised* protocol. The results of Section 4 are measured with the optimised protocol, and Section 4.5 compares it with the basic one. The changes alter only how the weights are committed and checked, which positions of the last block are proved for a next-token output, and how messages are sent. Theorem 3.2 then holds in the form

$$\varepsilon \leq \sum_j \left[p^{-r} + \prod_{i < t_j} \frac{m_j - 1 - i}{n_j - i} \right] \quad (3)$$

over the encoded matrices j (a looked-up embedding table adds no term), with message length m_j , codeword length n_j and t_j distinct opened columns. This product bounds the chance that all t_j positions fall among the at most $m_j - 1$ where two distinct codewords agree. (i) *Columns and codes.* Each Merkle tree opens the fewest distinct columns that keep the term of each of its matrices below $2^{-\beta}$, and matrices with codewords of equal length and similar column counts share one tree and one multiproof. A public *planning rule* picks, among six candidate rules, the codeword lengths and layouts that send the fewest bytes per query besides the claims, using at most twice as many encoded entries at setup as the basic protocol, or 2^{34} if that is more. LeNet-5’s matrices then share one tree of length 2^{18} and open 15 columns instead of 66 in each of five trees. (ii) *Transposed layout.* Committing the encoded rows $\hat{A}'$ of $A^\top$ lets the verifier combine positions instead of neurons: it draws r vectors $\chi' \in F_p^T$ (the T unit vectors when $T \leq r$), computes $z = Z\chi'$ and $w = [X; \mathbf{1}]\chi'$ itself for each, and checks $\text{Enc}(z)_c = (w^\top \hat{A}')_c$ on t opened columns of k entries. Since $w^\top \hat{A}' = \text{Enc}(Aw)$, a wrong claim passes only if all r products $(Z - A[X; \mathbf{1}])\chi'$ vanish or all t columns fall where two distinct codewords agree; in Eq. (3) its m_j is the number of rows of the matrices it stacks. No folded row is sent, and the opening shrinks from $rk + tN$ to tk field elements, which pays off for wide layers such as GPT-2’s output layer ($N=50,257$, $k=768$); projections of one input (query, key and value; gate and up) may share one such matrix. (iii) *Decoders.* Embedding tables without a bias are committed as Merkle trees over their rows, as in CommitLLM [6], so a wrong row needs a hash collision. For a next-token output, both parties compute the last block’s keys and values at every position and the rest of that block at the last position only, which gives the same logits. (iv) *Compact encoding.* Claims travel at 16–19 bits instead of 32 and field elements at 31; the verifier decodes before checking, and Fiat-Shamir absorbs the encoded bytes. The resulting plan (lengths, layouts, groups) follows from public shapes and is part of C_M . In K and K_{pre} , which open no columns, the optimised protocol keeps (iii), with the verifier reading the embedding rows itself, and (iv). Separately, *verifier engineering*, which changes no proof byte, makes the verifier evaluate $\text{Enc}(u_\ell)$ only at the opened positions, batch its exact products and compute attention in int32, and lets it stream the claims to a GPU. The prover of

the released code also computes attention in int32, which speeds up its forward pass.

4 Results

[Red: basic-protocol placeholder (red -: no basic run), to be replaced by the second optimised run.]

4.1 Experimental setup

Models. LeNet-5 (MNIST) and VGG-11, VGG-16 and ResNet-18 (CIFAR-10) are the CNN architectures that zkCNN, ZKML and Bionetta report on. Some published versions are smaller variants (ZKML’s ResNet-18 has 281K parameters), so Table 3 compares only models that match ours. A 224×224 dog-vs-cat ResNet-18 trained on ImageNet classes stands in for the unreleased model of [2], and a dog-vs-squirrel one for its substitute $\tilde{M}$; the MNIST MLP (784–512–256–10) is the network attacked in Section 4.2. We trained all of them, and int8 quantisation changed accuracy by at most 0.4 points. For language models we use the shapes of GPT-2, OPT-125M to OPT-13B, Llama-2-7B/13B and Qwen3-4B with random int8 weights. The costs depend only on the shapes, except the size of the compact encoding of Section 3.6, which depends on the values; we measure it on the same random weights, so it may differ for real checkpoints. **Every decoder up to 13B is built in full, at up to 2,048 tokens.**

Real weights. To check that the integer graph is a usable language model, we load the real OPT-125M, OPT-1.3B and OPT-6.7B checkpoints into the same graph, with one extra bias on the LM head, left over from folding the final LayerNorm into it. On 4,064 tokens of WikiText-2 their perplexity is 73.8, 37.4 and 36.0, against 64.7, 34.9 and 27.1 in fp32. These values depend on two public constants, the gain G of the integer normalisation (4, 3 and 2) and the activation-calibration percentile; the $G=32$ of our cost runs clips OPT’s outlier features and gives perplexities of 326 to 23,849. We chose both on the evaluation text, so these perplexities are slightly optimistic.

Hardware and baselines. The prover is one NVIDIA L40S (48 GB) and the verifier 8 threads of an AMD EPYC 9334; times are medians over a run’s queries. Of the 30–70B shapes only 1- and 2-block builds fit. From them we extrapolate proof sizes but not times; **where a full build exists, the extrapolated size is within 0.01% of it.** Published costs are taken as reported on their own hardware, each checked against its source; for the path protocol we compute exact detection probabilities and bytes on our graphs and take its time per path from Anchuri et al. The basic protocol was measured at commit 9401431, and a second run of it on an RTX 2080 Ti reproduced all 1,978 hardware-independent values. The optimised protocol was measured with the released code (commit c8be5eb) on the same machines; that code reproduces the basic proofs byte for byte. All tables and figures are generated from the released raw measurements.

4.2 Breaking the path protocol

Reproduction. Our implementation behaves as Anchuri et al. describe. On MNIST versions of four of their substitute-model settings (three from their experiments, one from their motivation),

honest provers were accepted in all 3,000 runs of each. Substitute models were caught in 99.8–100% of runs, including an 8-bit quantised copy that agrees with M on all 150 evaluated queries. Without the prescribed tolerance of 10^{-4} , an honest prover is accepted in 21–37% of runs, because a layer-wide matrix product and a single-neuron dot product round differently.

Attacks on MNIST. We compute each attack’s exact probability of passing one path by dynamic programming over the paths. The forgeries of [2] are its substitute model, the inverse-transform and logit-swap forgeries of its Appendices E and F, and the gradient reconstruction of its Section 7.2. They corrupt hundreds of the MLP’s 778 trace nodes and are always caught, while ours leave 1–14 inconsistent nodes. One tampered neuron is accepted with the predicted probability $1 - 1/N$ (99.6–99.8%). Triggered by a 3×3 bright patch, the backdoor returns the honest output on every clean query and succeeds on every triggered query not already in the target class. It passed 99.96% of 2,300 end-to-end runs, within sampling error of the predicted 99.8%. Keeping each tampered value below its neuron’s 95th percentile takes about five neurons and still passes 99% of the time under uniform sampling. With 250 paths, the single-neuron attack is accepted 61% of the time.

Other samplers. Against an adversary who knows the rule, uniform sampling detects 0.20% of attacks, a static importance score (the total magnitude of a neuron’s outgoing weights) 0.11%, and gradient saliency, which a verifier cannot compute without the model, 0.016%. The contribution-weighted sampler detects nothing: zeroing about 14 of the 512 neurons flips the prediction, and end-to-end runs accepted 2,000 of 2,000 attacks. Zero is also these neurons’ most common value (65% of inputs), so activation statistics do not flag the attack. A floor $\delta = 1$ raises detection to 2% (10× uniform); other floors give 0.9–6×.

Larger models. On every CNN and every test query, changing one penultimate neuron suffices to flip the prediction. One path catches it with probability 1/84 (LeNet-5) to 1/512 (VGG, ResNet), and a change to the least-visited neuron with probability as low as 1/28 million. This includes the dog-vs-cat ResNet-18, a binary classifier, for which Anchuri et al. argue that their analysis extends to full soundness because the adversary can only flip the output. Reaching 2^{-40} takes 2,316–14,182 paths, which open the whole model and trace (Figure 3). On Llama-2-7B one path detects the attack with probability 1/11,008, and 2^{-40} needs about 305,000 paths (≈ 13.7 GB with the fp16 weights of [2], at 64 tokens).

A real LLM. On the real OPT-6.7B (fp16), the attacker ranks a few FFN neurons of layers 8, 16, 24 and 31 at the last position, by gradient and by their direct effect on the logits. It then finds the smallest change to one of them that changes the next token. One neuron sufficed for all 40 prompts. A fixed-target backdoor is more constrained. Because of the final LayerNorm, raising one neuron’s activation far enough forces a single token, the neuron’s *saturation token*, so the last layer’s 16,384 neurons can force only 6,187 of the 50,272 tokens. The backdoor succeeds for a reachable target (“back”: 40/40 triggered prompts, clean prompts unchanged) and fails for “hacked” (0/40). A per-neuron range check would flag its values, which exceed the neurons’ natural range, but the path protocol has none; one path detects the backdoor with probability 1/16,384, and 2^{-40} needs 454,248 paths.

Optimised at 128 bits sends less than paths at 40 bits

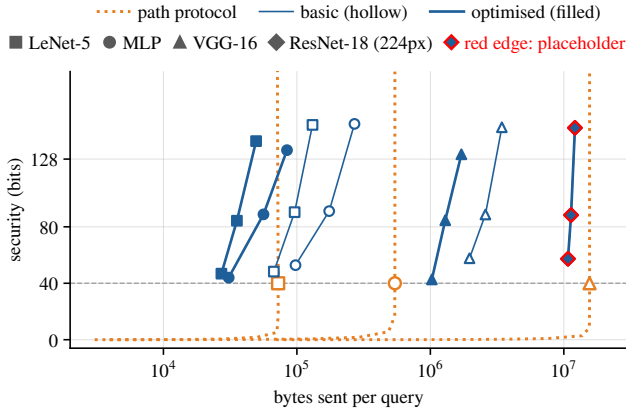


Figure 3: Security against bytes sent: our proofs buy far more security per byte than the paths

4.3 Soundness in practice

In the optimised runs the verifier accepted all 4,380 honest queries and rejected all 4,973 attacks. The 4,950 image-model attacks, at $\lambda=40$ with committed weights, were one changed value, one changed penultimate neuron, one changed logit, 1% of the weights perturbed, the substitute $\bar{M}$, and two forgeries that each pass every check but one; 3,300 of them were on commitments built under the planning rule. Every language-model run added +1 to one middle-layer pre-activation (23 attacks, 0 of them on 1- and 2-block builds of the 30–70B shapes), and for five models from OPT-1.3B to Llama-2-13B we also ran the image-model attacks that apply to them. Freivalds’ check rejected 3,909 attacks and the column check against the code 914, including every forged folded row and every tampered claim of a layer in the transposed layout; the Merkle check rejected all 150 forged columns, so each of the three checks is needed. The basic protocol’s run accepted all 8,709 honest queries and rejected all 2,895 attacks, 804 of them on language models.

Figure 3 plots security against the bytes sent per query for the image models, each with its own marker, orange for its paths and blue for our protocols. Security, in bits, is $-\log_2$ of the probability that a wrong output is accepted: for the paths, that the single-neuron attack is missed, and for ours, the proven bound at $\lambda=40, 80$ and 128. The paths reach 40 bits only by opening the whole model and trace (the vertical lines). The optimised protocol lands close to its target (43–57 bits at $\lambda=40$ and 131–150 at $\lambda=128$) and, on every model, reaches 2^{-128} with fewer bytes than the paths need for 2^{-40} . The basic protocol overshoots its target because r and t are rounded up (48–58 and 150–153 bits), and at $\lambda=40$ it sends 1.1–8.3 $\times$ fewer bytes than the paths need for 2^{-40} .

4.4 Cost

Image models. Table 1 gives the cost of one query to the optimised protocol at $\lambda=128$ (acc.: the int8 model’s accuracy). With committed weights the prover needs 3.8–27.4 ms per query and the verifier 1.7–175 ms. The proof, 49.5 kB–12.1 MB, is 1.3–9.3 $\times$ smaller

Table 1: Image models: proving takes milliseconds

Model	Params	acc.	committed (C)			K_{pre}	
			prove	verify	proof	verify	proof
MLP (MNIST)	536K	97.7%	3.8 ms	1.7 ms	84.3 kB	0.5 ms	3.1 kB
LeNet-5 (MNIST)	62K	99.3%	5.3 ms	2.4 ms	49.5 kB	1.7 ms	26.1 kB
VGG-11 (CIFAR)	9.8M	91.1%	12.9 ms	11.9 ms	1.0 MB	11.8 ms	610 kB
VGG-16 (CIFAR)	15.2M	93.0%	17.3 ms	14.9 ms	1.7 MB	19.4 ms	1.1 MB
ResNet-18 (CIFAR)	11.2M	94.2%	20.6 ms	24.7 ms	2.3 MB	46.5 ms	2.5 MB
ResNet-18 (224px) [†]	11.2M	95.0%	27.4 ms	175 ms	12.1 MB	105 ms	9.9 MB

[†]Our ImageNet-trained counterpart of the classifier of [2].

Table 2: Language models: seconds to prove, but proofs grow with the prompt

Model	Params	committed weights (C)				K _{pre}	
		prove	verify	GPU	proof	verify	proof
64-token prompts							
GPT-2	124M	80.2 ms	92.0 ms	427 ms	14.8 MB	137 ms	21.8 MB
Qwen3-4B	4.0B	1.9 s	4.6 s	–	659 MB	1.6 s	284 MB
Llama-2-7B	6.7B	1.7 s	1.1 s	3.8 s	329 MB	1.5 s	349 MB
Llama-2-13B	13.0B	4.6 s	6.7 s	–	1.2 GB	2.7 s	547 MB
2,048-token prompts							
OPT-125M	125M	1.1 s	14.2 s	691 ms	733 MB	12.7 s	692 MB
OPT-1.3B	1.3B	5.7 s	78.9 s	3.0 s	3.8 GB	81.2 s	3.7 GB
OPT-6.7B	6.7B	16.2 s	2.7 min	6.4 s	10.1 GB	2.7 min	9.7 GB
Llama-2-7B	6.7B	17.6 s	2.8 min	6.9 s	11.6 GB	2.7 min	11.2 GB
Llama-2-13B	13.0B	27.1 s	3.7 min	–	18.1 GB	3.7 min	17.5 GB

than the int8 weights for the MNIST and CIFAR models; for the 224-pixel ResNet, whose claims grow with the number of pixels, it is about their size.

Language models. Table 2 proves the next-token logits of one prompt at $\lambda=128$ with random int8 weights; its GPU column runs our verifier on the GPU. With a 64-token prompt GPT-2 is proved in 80.2 ms and Llama-2-7B in 1.7 s, and Llama-2-7B’s 329 MB proof is about 21 $\times$ smaller than its int8 weights. At 2,048 tokens Llama-2-7B is proved in 17.6 s, but the proof grows to 11.6 GB and CPU verification to 2.8 min. The proof grows because the claims grow with the prompt (the K_{pre} proof, which holds only claims, is almost as large), and the verifier because it recomputes attention over all token pairs; in setting C a GPU verifier is 21–26 $\times$ faster there and 1.1–1.2 $\times$ at 64 tokens. Prompts whose claims are all sent before the challenge can share one set of challenges and columns; in batches of eight 64-token prompts, Llama-2-7B’s prover then needs 0.40 s and its proof 401 MB per prompt. For Llama-2-70B the extrapolated proof at 64 tokens is 3.7 GB, 18 $\times$ smaller than its int8 weights.

Figure 4 places these runs among the published ones, with our verifier on 8 CPU threads and the published systems as reported on their own hardware; Table 3 compares them on matched models. Part of the prover gap is our GPU, since all the zkSNARKs except zkLLM ran on CPUs. The orange cross is one path of [2], as reported on an RTX 3090.

Prover. Ours is faster than every zkSNARK prover on its matched model, by **30×** (zkLLM, Llama-2-13B) to **32,000×** (ZKML, VGG-16). Part of this is hardware, so we also ran zkLLM’s public code on our L40S. Its demo proves the components of one layer separately and reports no verifier time, so we time one layer (median of three runs on each of two layers) and multiply by the number of layers. The demo lacks zkLLM’s cross-layer optimisation and the attention output projection, and its attention step writes no output, so we count the layer’s other skip connection twice instead of the one after it. For Llama-2-7B at 2,048 tokens this gives about 844 s, **48×** our **17.6 s**. We do not use the demo for Llama-2-13B, whose time per layer grows out of proportion (2.5× the 7B figure).

Verifier and proof. Our verifier is slower only than ZKML’s on VGG-16 (1.9×), **zkLLM’s at 2,048 tokens (2.0–3.4×)** and Maverick’s (1.2×). Maverick’s 124.5 ms is its 87.1 ms of matrix checks plus the 37.4 ms in which its client replays the non-linear operations, as our verifier does (the client’s whole online time is 260.2 ms); ours takes 155.5 ms, 43.4 ms of it decoding the compact encoding. Without the compact encoding our Qwen3-4B run beats Maverick on all three costs (117 against 354.5 ms, 111.0 against 124.5 ms and 35.19 against 36.08 MB); our prover, however, runs on the GPU and Maverick’s on 64 CPU threads. Maverick also draws a fresh challenge per query, whereas K_{pre} reuses a secret one. On LeNet-5 and on GPT-2 with a 64-token prompt the non-interactive optimised protocol wins on all three costs: 5.4 ms, 2.6 ms and 63.3 kB against zkCNN’s 441 ms, 5.8 ms and 71.3 kB, and 91 ms, 105 ms and 16.6 MB against DeepProve’s 34.2 s, 1.35 s and 21.7 MB. zkCNN’s verifier ran on one core and ours on 8 threads, and zkCNN’s protocol has a zero-knowledge variant, which ours lacks. DeepProve’s figures are for its transparent (BaseFold) configuration, whose proof certifies all 64 positions where ours certifies the next-token logits; its HyperKZG configuration, which needs a structured reference string, sends 9.4 MB. Elsewhere our proof is larger, by 3.6–180× against the other short-input proofs and by **5,200–96,000×** against zkLLM’s at 2,048 tokens.

5 Discussion

Implications. The path protocol catches a provider who runs the wrong model, but not one who runs the committed model and edits the answer, and no sampling rule closes that gap in the worst case. Full soundness, however, does not need a zkSNARK prover: one random linear combination per weight layer gives a $2^{-\lambda}$ guarantee with no trusted setup and no assumption about how models behave. For MLPs and CNNs, which have no attention, the client verifies in milliseconds on a CPU and the proof is usually smaller than the model, so even a modest client can check every answer. For language models the verifier recomputes attention, so the protocol suits a gateway or an auditor with a GPU and a fast link rather than a phone; it works best for large models with short prompts, where the proof is far smaller than the model.

Limitations. (i) The proof grows with the activations; **at 2,048 tokens it is mostly claims and gigabytes for 7B models**, against zkLLM’s 157–183 kB. (ii) The verifier’s cost grows with the square of the prompt length: at 2,048 tokens the CPU verifier needs up to 3.7 min, and **the GPU verifier is 2.0–3.4× slower than zkLLM’s**. (iii) The provider must serve the integer model, which changed CNN accuracy by at most 0.4 points and raised the perplexity of real OPT checkpoints by 7–33% (constants chosen on the evaluation text); the LLM costs were measured with random weights. (iv) Like the path protocol we assume an honest weight commitment, which the planning rule’s longer codewords make up to 19× slower to compute; a malformed one would need a proximity argument as in Ligerio. (v) The opened columns reveal random linear combinations of each weight row, so the protocol is not zero-knowledge. (vi) We prove one forward pass, without a KV cache. (vii) The prototype is Python, and the same-GPU zkLLM comparison rests on its demo.

Future work. The claims, which the compact encoding roughly halves, could be replaced by a sum-check argument for the non-weight operations. Checking the attention products instead of re-computing them, with Freivalds’ check on claimed scores or a sum-check argument as in zkLLM [31], could make the verifier much faster at long prompts, at some cost in proof size. A native verifier and a KV cache would also help, and per-channel normalisation gains may narrow the perplexity gap at no proving cost. Before any wider publication we will share the attack with the authors of [2].

6 Conclusion

A single tampered neuron defeats the path protocol of [2], on an MNIST network and on the real OPT-6.7B, and no sampling rule repairs it in the worst case. Checking every layer with Freivalds’ algorithm against a Reed–Solomon/Merkle commitment does: a wrong output is accepted with probability at most $2^{-\lambda}$, and the protocol rejected all 2,895 attacks on the basic protocol and all **4,973** on the optimised one. Its prover is one to four orders of magnitude faster than the zkSNARK provers, and on LeNet-5 and 64-token GPT-2 prompts it also verifies faster and sends less than zkCNN and DeepProve, with the caveats of Section 4.6. Long prompts, **where the proof is mostly claims**, remain its weak point.

7 Contributions

[To be completed by the team.] Ofek Lavi: [...]. Erel Barzilay: [...]. Edo Kadosh: [...].

References

- [1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. 2017. Ligerio: Lightweight Sublinear Arguments Without a Trusted Setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2087–2104. doi:10.1145/3133956.3134104
- [2] Pranay Anchuri, Matteo Campanelli, Paul Cesaretti, Rosario Gennaro, Tushar M. Jois, Hasan S. Kayman, and Tugce Ozdemir. 2026. Towards Verifiable AI with Lightweight Cryptographic Proofs of Inference. In *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 955–977. doi:10.1109/SaTML68715.2026.00058
- [3] Arasu Arun, Adam St. Arnaud, Alexey Titov, Brian Wilcox, Viktor Kolobaric, Marc Brinkmann, Oguzhan Ersoy, Ben Fielding, and Joseph Bonneau. 2025. Verde: Verification via Refereed Delegation for Machine Learning Programs. arXiv:2502.19405.
- [4] Wyatt Benno, Alberto Centelles, Antoine Douchet, and Khalil Gibran. 2026. Jolt Atlas: Verifiable Inference via Lookup Arguments in Zero Knowledge. arXiv:2602.17452.
- [5] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. 2019. Fiat-Shamir: From Practice to Theory. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)*. ACM, 1082–1090.
- [6] Federico Carrone, Diego Kingston, Manuel Puebla, and Mauro Toscano. 2026. CommitLLM: A Commit-and-Audit Protocol for Open-Weight LLM Inference. Preprint, <https://github.com/lambdaclass/CommitLLM>.
- [7] Bing-Jyue Chen, Lilia Tang, and Daniel Kang. 2025. ZKTorch: Compiling ML Inference to Zero-Knowledge Proofs via Parallel Proof Accumulation. arXiv:2507.07031.
- [8] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. ACM, 560–574. doi:10.1145/3627703.3650088
- [9] Ziqun Chen, Ming Wu, Michael Heinrich, Jason Zeng, Huiying Lan, Tianwei Zhang, and Rui Tan. 2026. Communication-Efficient Verifiable Attention for LLM Inference. arXiv:2606.16352. Presents VeriAttn.
- [10] KD Conway, Cathie So, Xiaohang Yu, and Kartin Wong. 2024. opML: Optimistic Machine Learning on Blockchain. arXiv:2401.17555.
- [11] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network

- Inferences. Cryptology ePrint Archive, Paper 2021/087. <https://eprint.iacr.org/2021/087>
- [12] Boyuan Feng, Zheng Wang, Yuke Wang, Shu Yang, and Yufei Ding. 2024. ZENO: A Type-based Optimization Framework for Zero Knowledge Neural Network Inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 450–464. doi:10.1145/3617232.3624852
 - [13] Amos Fiat and Adi Shamir. 1986. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO '86 (LNCS, Vol. 263)*. Springer, 186–194.
 - [14] Rusins Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *Information Processing 77, Proceedings of the IFIP Congress*. North-Holland, 839–842.
 - [15] Nicolas Gailly, Ismael Hishon-Rezaizadeh, Tianyi Liu, Nicholas Mainardi, Dimitrios Papadopoulos, Charalampos Papamanthou, Christodoulos Pappas, Shrawan Srinivasan, Zack Youell, and Yupeng Zhang. 2026. DeepProve: Verifiable End-to-End Large Language Model Inference. Cryptology ePrint Archive, Paper 2026/1112. Minor revision at ACM CCS 2026 (ePrint publication info).
 - [16] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
 - [17] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2008. Delegating Computation: Interactive Proofs for Muggles. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 113–122.
 - [18] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. 2023. Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS. In *Advances in Cryptology – CRYPTO 2023*. Springer, 193–226. doi:10.1007/978-3-031-38545-2_7
 - [19] Kanav Gupta, Jonathan Katz, and Ian Miers. 2026. Private and Verifiable Outsourcing of Open-Weight LLM Inference. Cryptology ePrint Archive, Paper 2026/1849.
 - [20] Justin Holmgren. 2019. On Round-By-Round Soundness and State Restoration Attacks. Cryptology ePrint Archive, Paper 2019/1261.
 - [21] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.
 - [22] Kyeongtae Lee, Byeongkyu Han, Jihye Kim, and Hyunok Oh. 2026. LAMP: Linear Verification of Matrix Multiplication via Proximity Testing. Cryptology ePrint Archive, Paper 2026/1571.
 - [23] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. doi:10.1109/TDSC.2023.3348760
 - [24] Youki Lim and Sam Yong. 2026. Seal, Then Sample: Sampled Layerwise Proofs for Verifiable LLM Inference from GPT-2 to 70B. arXiv:2609.27367.
 - [25] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2968–2985. doi:10.1145/3460120.3485379
 - [26] Ben Merbaum, Mohammad Amin Raeisi, Wenhao Wang, Charalampos Papamanthou, Katerina Sotiraki, and Fan Zhang. 2026. Maverick: Private and Verifiable LLM Inference Made Practical via Matrix-Vector Multiplication Delegation. arXiv:2609.10264.
 - [27] Ralph C. Merkle. 1987. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO '87 (LNCS, Vol. 293)*. Springer, 369–378.
 - [28] Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghouar, Manveer Basra, Max Ryabinin, and Johannes Hagemann. 2025. TOPLOC: A Locality Sensitive Hashing Scheme for Trustless Verifiable Inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 47196–47211.
 - [29] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. 2025. zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference. In *34th USENIX Security Symposium*. USENIX Association, 2045–2063.
 - [30] Irving S. Reed and Gustave Solomon. 1960. Polynomial Codes over Certain Finite Fields. *J. Soc. Indust. Appl. Math.* 8, 2 (1960), 300–304.
 - [31] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 4405–4419. doi:10.1145/3658644.3670334
 - [32] Yifan Sun, Yuhang Li, Yue Zhang, Yuchen Jin, and Huan Zhang. 2024. SVIP: Towards Verifiable Inference of Open-source Large Language Models. arXiv:2410.22307.
 - [33] Florian Tramèr and Dan Boneh. 2019. Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware. In *International Conference on Learning Representations (ICLR)*.
 - [34] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium*. USENIX Association, 501–518.
 - [35] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. 2025. zkPyTorch: A Hierarchical Optimized Compiler for Zero-Knowledge Machine Learning. Cryptology ePrint Archive, Paper 2025/535.
 - [36] Jiaqi Xue, Yifei Zhao, Mengxin Zheng, Fan Yao, Yan Solihin, and Qian Lou. 2025. Securing Transformer-based AI Execution via Unified TEEs and Cryptoprotected Accelerators. arXiv:2507.03278. Presents TwinShield.
 - [37] Jianzhu Yao, Hongxu Su, Taobo Liao, Zerui Cheng, Huan Zhang, Xuechao Wang, and Pramod Viswanath. 2026. TAO: Tolerance-Aware Optimistic Verification for Floating-Point Neural Networks. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. ACM, 1515–1532. doi:10.1145/3767295.3803612
 - [38] Dmytro Zakharov, Oleksandr Kurbatov, Artem Sdobnov, Lev Soukhanov, Yevhenii Sekhin, Vitalii Volovyk, Mykhailo Velykodnyi, Mark Cherepovskyi, Kyrlo Baibula, Lasha Antadze, Pavlo Kravchenko, Volodymyr Dubinin, and Yaroslav Panasenko. 2025. Bionetta: Efficient Client-Side Zero-Knowledge Machine Learning Proving. arXiv:2510.06784.
 - [39] Zkonduit. 2026. EZKL: An Engine for Doing Inference for Deep Learning Models and Other Computational Graphs in a zk-SNARK. <https://github.com/zkonduit/ezkl>. GitHub repository, accessed September 2026.